

Heap Data Structure

Kanishk Goel

14 November 2025

1 Heaps

Heaps Basic Operations
Insert: given a heap H and a new object x , add x to H . ($O(\log n)$)
ExtractMin: given a heap H , remove and return from H an object with the smallest key. ($O(\log n)$)
FindMin: given a heap H , return an object with the smallest key. ($O(1)$)
Heapify: given objects $x_1, x_2, \dots, x_n$, create a heap containing them. ($O(n)$)
Delete: given a heap H and a pointer to an object x in H , delete x from H . ($O(\log n)$)

2 HeapSort

HeapSort
Input: An array of n numbers, in arbitrary order. Output: An array of the same numbers, sorted from smallest to largest.
$H \leftarrow$ empty heap for $i = 1$ to n do Insert $A[i]$ into H for $i = 1$ to n do $B[i] \leftarrow$ ExtractMin from H

3 Median Maintenance

Question: Given a sequence of numbers, present the median of the numbers every time a new number is given.

Solution: Keep a Min Heap H_1 and a Max Heap H_2 , keep both the heaps balanced, i.e having equal number of elements for $2k$ elements and for odd iteration $(2k + 1)$, a discrepancy of 1.

Idea is that the maximum element of H_2 and/or minimum element of H_1 will be the median. Therefore, for i^{th} iteration, insert the element based on comparison with the previous median. To keep the balancing property, only edge case is that of a new element coming in at $2k + 1$ elements total. Here, one heap might have $k + 1$ and other $k - 1$ elements. So, just extract min/max based on the heap type and insert it into the other heap.

Runtime: $O(\log i)$ for each i^{th} iteration, since a constant number of heap operations are being done every iteration.